

Energy Efficiency in Railway Scheduling

Paul Tennigkeit and Georgi Abshilava

University of Potsdam, Germany
{tennigkeit, abshilava}@uni-potsdam.de

1 Introduction

Railway networks these days face continuous pressure to optimize operations and reduce delays, while also operating energy efficiently. Within the domain of railway scheduling, finding conflict-free routes for multiple agents on a shared grid is a complex combinatorial challenge. To study these dynamics, researchers often utilize the Flatland environment, which serves as a benchmarking tool for multi-agent coordination. While much of the existing research uses Reinforcement Learning to navigate Flatland’s grid worlds, our project adopts a declarative approach. We utilize Answer Set Programming (ASP) to formulate the scheduling task as a series of logical constraints, allowing the solver to rigorously explore the combinatorial space for optimal routing.

Traditional pathfinding in Flatland generally focuses on a single objective: navigating agents from a starting point to a destination in the shortest amount of time. However, real-world railway logistics demand the optimization of secondary, physically grounded metrics. In this paper, we extend Flatland’s routing logic by introducing energy efficiency as a core constraint. We model the physical realities of train operations by calculating energy consumption dynamically based on individualized vehicle mass, motor efficiency (electric, diesel, hybrid, and hydrogen), and kinematic states such as waiting, cruising, and turning. By treating railway scheduling as a multi-objective optimization problem, we aim to evaluate how the introduction of heavy energy penalties influences the routing choices made by the ASP solver.

The remainder of this report is structured as follows. In Section 2, we introduce the baseline ASP encoding required for basic vehicle routing and collision avoidance within Flatland. We then detail the

mechanics of our energy encoding, explaining how physical traits and movement penalties are mathematically integrated into the logic. In Section 3, we present the results of our experiments, analyzing how the solver navigates the trade-offs between arrival time and fuel efficiency. Finally, in Section 4, we conclude on our findings regarding the computational complexity of the encoding and suggest future directions for research in multi-objective railway scheduling.

2 Encoding Energy Efficiency in ASP

2.1 What is Flatland?

Flatland is a framework for simulating railway networks. A Flatland environment consists of a two dimensional plane made up of different types of cells, some of which containing different types of interconnected tracks, and agents moving around on them from a starting point to an end point. Agents have an earliest departure and arrival time, and a pre-defined speed, represented by an inverse value. To limit the time of the simulation, a global time frame is set.

2.2 Baseline Encoding

The baseline ASP encoding handles all essential train and environment behavior, including defining legal transitions and movements, possible directions, and actions.

Directions There are four possible directions agents can go and point towards: North, south, east and west. Each is defined as a fact:

```
dir(n). dir(s). dir(e). dir(w).
```

Movements Each possible movement is defined as a fact:

```
moving(move_forward).  
moving(move_left).  
moving(move_right).
```

`delta()` defines what it means to "move" in each direction:

% The origin is located in the top left in Flatland, and
% the coordinates increment downwards

```
delta(A,s,1,0) :- moving(A).  
delta(A,n,-1,0) :- moving(A).  
delta(A,e,0,1) :- moving(A).  
delta(A,w,0,-1) :- moving(A).
```

Transitions In order to represent different types of tracks, `transitions()` predicates are defined for each:

```
% ID : track ID  
% Dir1: pointing direction of agent  
% Act : associated action with movement towards exit  
% Dir2: exit direction  
transition(ID, Dir1, Act, Dir2).
```

Due to the way this predicate is built, each track type is defined multiple times depending on its number of exit directions (e.g. straight tracks have two possible exits, north and south, and are, therefore, defined twice for each possible exit).

Actions The `action()` predicate is used as an interface between the Flatland visualization engine and the Clingo logic:

```
% train(ID): A train with an ID  
% A : An action  
% T : The time step action A is taken  
action(train(ID),A,T) :- chosen(ID,T,A,_,_,_,_,_).  
action(train(ID),move_forward,Dep) :- start(ID,_,Dep,_),  
                                         Dep > 0.  
  
action(train(ID),wait,T) :- start(ID,_,Dep,_),  
                               Dep > 0,  
                               T = 0..Dep-1.
```

Collision Constraints Like in real life, collisions between trains are to be avoided. To achieve this, the following constraints are employed:

```
% Trains can not occupy the same cell at the same time
```

```
:- stay(ID1,X,Y,T),
   stay(ID2,X,Y,T),
   ID1 < ID2.
```

% Trains can not swap their position

```
:- stay(ID1,X1,Y1,T),
   stay(ID2,X2,Y2,T),
   stay(ID1,X2,Y2,T+1),
   stay(ID2,X1,Y1,T+1),
   ID1 < ID2.
```

Reachability Constraints To simulate movement across the grid over time, a train's initial departure parameters are first converted into a `position()` state:

```
position(ID,(X,Y),Dep+1,Dir) :- start(ID,(X,Y),Dep,Dir).
```

From any current position, valid future states are derived using the `reachable()` predicate by checking cell track transitions, movement coordinate deltas, and the train's speed:

```
reachable(ID,T,A,X2,Y2,T2,Dir2) :- A != wait,
                                     position(ID,(X,Y),T,Dir1),
                                     cell((X,Y),Cell),
                                     transition(Cell,Dir1,A,Dir2),
                                     delta(A,Dir2,DX,DY),
                                     X2 = X + DX,
                                     Y2 = Y + DY,
                                     speed(ID,S),
                                     T2 = T + S,
                                     time(T2).
```

```
reachable(ID,T,wait,X,Y,T+1,Dir) :- position(ID,(X,Y),T,Dir),
                                     time(T+1).
```

At each discrete time step, a choice rule ensures exactly one valid transition is taken until the destination is reached:

```
1 { position(ID,(X2,Y2),T2,Dir2)
   : reachable(ID,T,A,X2,Y2,T2,Dir2) } 1
:- position(ID,(_,_),T,_), not reached(ID,T).
```

In order to properly enforce the collision constraints, the time interval a train spends inside a specific cell must be tracked:

```
chosen(ID,T,A,X,Y,X2,Y2,T2) :- position(ID,(X,Y),T,_),
                                position(ID,(X2,Y2),T2,_),
                                reachable(ID,T,A,X2,Y2,T2,_).
```

Because trains can have an inverse speed greater than 1 (meaning they traverse a cell over multiple time steps), the `stay()` predicate marks every time step a cell is occupied between entering and leaving:

```
% for inverse speed !=1: define interval to stay in a cell
stay(ID,X,Y,T1) :- chosen(ID,T,_,X,Y,_,_,T2),
                    time(T1), T <= T1, T1 < T2.
```

Target Arrival A train successfully completes its journey when its current coordinates match its designated target destination:

```
reached(ID,T) :- position(ID,(X,Y),T,_),
                 end(ID,(X,Y),_).
```

To ensure valid and bounded schedules, a look-ahead integrity constraint terminates search branches where a train can physically no longer reach its end coordinates within its individual allowable arrival deadline:

```
% stop searching if train is not in time
% to reach its target anymore
:- position(ID,(X,Y),T,_),
   end(ID,(EX,EY),Arr),
   speed(ID,InvSpd),
   T + (|X-EX| + |Y-EY|)*InvSpd > Arr.
```

By evaluating the remaining Manhattan distance multiplied by the train's inverse speed directly against its scheduled deadline, this single rule simultaneously prevents late arrivals and prunes unviable paths early in the search space.

Schedule Optimization Once valid routing pathways are established, the solver optimizes the schedule by minimizing overall transit delays. To ensure that trains are evaluated only on their actual transit

time instead of the time spent idling at their destination after completion, the encoding isolates the exact time step T at which a train first reaches its end coordinates:

```
% the arrival times should be minimized
reached_earlier(ID,T) :- reached(ID,T),
                           reached(ID,T0),
                           T0 < T.
arrival(ID,T) :- reached(ID,T),
                  not reached_earlier(ID,T).
```

The `#minimize` statement aggregates the arrival times of all agents and assigns this objective a priority level of `@2`. This high priority ensures that in extended multi-objective encodings minimizing passenger schedule delay always takes precedence over secondary optimization metrics.

```
% minimize summed arrival times
#minimize { T@2, ID : arrival(ID,T) }.
```

2.3 Acceleration Mechanics

To model energy consumption realistically, we add acceleration mechanics to the baseline encoding.

Maximum and Initial Speeds The environment created for the experiments also defines the trains' speeds in form of `speed()` predicates, which, as mentioned in chapter 2.1, is interpreted as an inverse speed (e.g. a `speed` of 4 actually means, that a train would be 25% slower). To model acceleration, `speed()` is reinterpreted as `max_speed()`.

```
% interpret given train speeds as maximum speeds
max_speed(ID,S) :- speed(ID,S).
```

As a train starts moving, it initially has a speed of 0.25.

```
% trains start with a speed of 1/4
temp_speed(ID, Dep+1, 4) :- start(ID,(_,_),Dep,_).
```

Dynamic Speed Change The `temp_speed()` choice rules allow gradual speed changes while strictly respecting a train's physical limits. During each position change, a train can alter its inverse speed by a maximum of one step at a time.

```
1{
    temp_speed(ID,T2,S-1);
    temp_speed(ID,T2,S);
    temp_speed(ID,T2,S+1)
}1
:- chosen(ID,T,A,_,_,_,_,T2),
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 >= MS,
   S+1 <= 4.
```

To account for the upper and lower bounds of a train's speed capabilities, three additional choice rules handle edge cases: when a train is at its maximum speed, when it is at its minimum speed, or when both conditions apply simultaneously. In the latter case, trains are forced to maintain their current speed.

```
% Lower bound
1{
    temp_speed(ID,T2,S-1);
    temp_speed(ID,T2,S)
}1
:- chosen(ID,T,A,_,_,_,_,T2),
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 >= MS,
   S+1 > 4.

% Upper bound
1{
    temp_speed(ID,T2,S);
    temp_speed(ID,T2,S+1)
```

```

}1
:- chosen(ID,T,A,_,_,_,_,T2),
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 < MS,
   S+1 <= 4.

% In between
temp_speed(ID,T2,S)
:- chosen(ID,T,A,_,_,_,_,T2),
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 < MS,
   S+1 > 4.

```

Post-Wait Acceleration When a train decides to wait, it loses its momentum. The encoding dictates that after taking a `wait` action, a train must resume movement at the lowest possible speed:

```

% after waiting, you have to start with speed 1/4
temp_speed(ID,T2,4) :- chosen(ID,T,wait,_,_,_,_,T2).

% to be able to wait, you need to have speed 1/4
reachable(ID,T,wait,X,Y,T+1,Dir)
:- position(ID,(X,Y),T,Dir),
   time(T+1),
   temp_speed(ID,T,4).

```

Turning and Arrival Physics In real life, trains must decelerate before taking sharp turns or when pulling into a station. This rule enforces, that a train cannot take a turn at high speeds; it must slow down to an inverse speed of at least 3. Similarly, it must fully slow down to an inverse speed of 4 upon reaching its final target.

```

% you have to take a turn with inverse speed >= 3
:- position(ID,_,T,Dir1),
   temp_speed(ID,T,S),

```

```

    T2 = T + S,
    position(ID,_,T2,Dir2),
    Dir1 != Dir2,
    S < 3.

% you can only arrive at your goal if your inverse speed was 4 before
:- reached(ID,T2),
    temp_speed(ID,T1,S),
    T2 = T1 + S,
    S < 4.

```

Solver Heuristics To guide the solver’s search process through the expanded state space, the encoding introduces specific **#heuristic** directives to prioritize specific branches by penalizing waiting, scheduling trains one by one based on departure times, and instructing the solver to prefer paths where trains gain speed quickly. These heuristics make finding a solution around four times faster within our example environment (see figure 1), decreasing the computation time from 3072.10 seconds to 845.88 seconds:

```

% prefer to not wait
wait_position(ID,X2,Y2,T2,Dir2)
:- reachable(ID,T,wait,X2,Y2,T2,Dir2).

#heuristic position(ID,(X2,Y2),T2,Dir2)
    : wait_position(ID,X2,Y2,T2,Dir2). [-50,1]

% schedule trains one by one
#heuristic position(ID,(X,Y),T,Dir)
    : start(ID,_,Dep,_). [-Dep,1]

% prefer getting faster
#heuristic temp_speed(ID,T,S)
    : position(ID,_,T,_). [-S,1]

```

2.4 Energy Encoding Mechanics

The energy efficiency encoding adds new rules to represent the concept of energy and how different actions of the agents affect their usage and consumption of energy.

Cost Factors In real life, trains use a variety of motors that rely on different sources of energy, such as electric, hybrid, diesel or hydrogen power, which fundamentally affects how efficiently energy is consumed. The `pow_cost_fac()` predicate represents this baseline efficiency in the encoding by assigning an integer multiplier to each train based on its power type.

To then determine the true energy footprint of an agent, its mass must also be accounted for. This is handled by the `cost_fac()` rule, by multiplying a train's assigned cost factor, the PCF, by its defined weight `W`, resulting in a final, individualized cost factor `CF` for each train in the simulation:

```
% energy cost factor of trains based on power type + weight
pow_cost_fac(ID,1) :- pow_type(ID, electro).
pow_cost_fac(ID,2) :- pow_type(ID, hybrid).
pow_cost_fac(ID,3) :- pow_type(ID, diesel).
pow_cost_fac(ID,4) :- pow_type(ID, hydrogen).

cost_fac(ID,CF) :- pow_cost_fac(ID,PCF),
                   weight(ID,W),
                   CF = PCF * W.
```

The trains' masses and energy types are all set in the environment definition.

Energy Costs of Actions Depending how fast the train moves and what it does, it consumes different amounts of energy. In the encoding, energy is divided into three distinct kinematic states:

- **Moving Straight:** The energy consumed while moving forward is proportional to the square of the train's speed. The encoding calculates this using the formula $E = 100 \cdot CF \cdot \left(\frac{1}{s}\right)^2$. The multiplier of 100 is explicitly introduced to prevent precision loss from unnecessary rounding during the solver's integer calculations.

```

% Cost of moving straight
energy(ID,T2,E) :- action(train(ID),A,T),
                    A != wait,
                    position(ID,_,T,Dir),
                    temp_speed(ID,T,S),
                    T2 = T + S,
                    position(ID,_,T2,Dir),
                    cost_fac(ID,CF),
                    E = 100 * CF * 1/S * 1/S.

```

- **Turning:** When an agent goes on a curve, centrifugal forces demand additional work. This is modeled by applying a 10% penalty to the base straight-line movement cost, resulting in the calculation $E = 100 \cdot CF \cdot \left(\frac{1}{S}\right)^2 \cdot \frac{11}{10}$.

```

% Cost of turning
energy(ID,T2,E) :- position(ID,_,T,Dir1),
                    temp_speed(ID,T,S),
                    T2 = T + S,
                    position(ID,_,T2,Dir2),
                    Dir1 != Dir2,
                    cost_fac(ID,CF),
                    E = 100 * CF * 1/S * 1/S * 11/10.

```

- **Braking:** To simulate modern regenerative braking capabilities, agents actually recover energy when coming to a halt. The encoding models this as a 40% energy return based on the base speed squared, represented by the formula $E = -1 \cdot 100 \cdot \frac{4}{10} \cdot CF \cdot \left(\frac{1}{4}\right)^2$.

```

% "Cost" of braking
prev_wait(ID,T) :- action(train(ID),_,T),
                    time(T-1),
                    action(train(ID),wait,T-1).

```

```

energy(ID,T+1,E) :- action(train(ID),wait,T),
                        not prev_wait(ID,T),
                        cost_fac(ID,CF),
                        E = -1 * 100 * 4/10 * CF * 1/4 * 1/4.

```

Removal of Acceleration Heuristics As the heuristics increased the time of finding a solution, from 950.83 seconds up to 2600 seconds depending on which heuristic was active, we decided to remove them in the energy encoding.

2.5 Results

For all trial runs a single environment was used. It consists of a 15×25 network with two stations and the following two agents:

```

train(0).
start(0,(6,19),0,w).
end(0,(10,5),63).
speed(0,1).
pow_type(0, electro).
weight(0,2).

train(1).
start(1,(9,5),0,e).
end(1,(7,19),63).
speed(1,1).
pow_type(1, diesel).
weight(1,1).

```

The baseline encoding was only used to establish rules for bringing trains from a starting point to an end point with a proper behavior for a given environment. That goal was reached successfully: The baseline encoding successfully generated paths for all two agents from their starting point to their end points (see tables 1 and 2) in a reasonable time frame of 16.029 seconds.

With the baseline routing established, we can analyze how the physical constraints of the energy encoding influence the trains' behavior.



Fig. 1. Flatland environment used for all experiments

Train 0, defined as an electric train with a weight of 2, shows a significant acceleration on straight tracks, where it reaches its maximum inverse speed of 1. This impacts its power consumption, causing spikes of 200 units per timestep. However, the solver forces train 0 to slow down to an inverse speed of 3 before turning, which can be seen at timesteps 13, 36, 45 and 50 (see table 3), according to the turning constraints.

Train 1 is modeled as a diesel train with a weight of 1. This train’s route showcases the regenerative braking capabilities defined in the encoding: When train 1 departs from a waiting state, it successfully regains a small amount of energy, logged as -7 units, at timesteps 17 and 27. It reaches its top inverse speed of 2, where its energy consumption peaks at 75 units on straight tracks (see table 4).

The energy encoding managed to find a solution in 702.315 seconds (approximately 12 minutes). For this small environment with only two agents, this increase in computation time highlights the scalability limitations of integrating physical modeling, such as energy consumption, it being computationally expensive.

3 Conclusion

In this project, we extended the railway scheduling problem within the Flatland framework by modeling energy consumption using An-

swer Set Programming (ASP). Our results demonstrate that the ASP encoding accurately captures physical realities at the local level: energy costs dynamically scale based on vehicle mass, power type, and specific kinematic states.

However, our current ASP encoding exhibits limitations regarding computational scalability. The introduction of complex energy penalties expands the search space significantly, particularly because the solver must evaluate numerous kinematic combinations across extended time horizons.

We consider several extensions and modifications for future work based on our findings:

- **Algorithmic Improvements:** To manage the exponential growth of the search space on larger grids, future research could employ different solving techniques. Progressively extending the planning horizon could help approximate optimal energy solutions more efficiently without grounding the entire time horizon at once.
- **Environmental Physics:** With dynamic acceleration and momentum mechanics now established, future iterations could incorporate track elevation and gradients, adding another layer of realism to how trains consume power on inclines or recover energy on declines.
- **Alternative Optimization Strategies:** Adjusting or combining the minimize directives and/or the heuristics could allow the solver to actively evaluate and choose longer, more energy-efficient detours rather than strictly defaulting to the fastest geometric route.

Overall, we provide a robust proof of concept that ASP can effectively model and optimize physical constraints in railway logistics, laying the groundwork for future research into multi-objective routing.

A Tables for Baseline Encoding

Table 1. Path taken by agent 0 in the baseline encoding

agent	timestep	position	direction	status	given_command
0	1	(6, 19)	w	waiting	move_forward
0	2	(6, 18)	w	ready to depart	wait
0	3	(6, 18)	w	ready to depart	wait
0	4	(6, 18)	w	ready to depart	wait
0	5	(6, 18)	w	ready to depart	wait
0	6	(6, 18)	w	ready to depart	wait
0	7	(6, 18)	w	ready to depart	wait
0	8	(6, 18)	w	ready to depart	wait
0	9	(6, 18)	w	ready to depart	wait
0	10	(6, 18)	w	ready to depart	wait
0	11	(6, 18)	w	ready to depart	wait
0	12	(6, 18)	w	ready to depart	wait
0	13	(6, 18)	w	ready to depart	move_forward
0	14	(6, 17)	w	moving	wait
0	15	(6, 17)	w	stopped	move_forward
0	16	(6, 16)	w	moving	move_forward
0	17	(6, 15)	w	moving	move_forward
0	18	(6, 14)	w	moving	move_forward
0	19	(6, 13)	w	moving	move_forward
0	20	(6, 12)	w	moving	move_forward
0	21	(6, 11)	w	moving	move_forward
0	22	(6, 10)	w	moving	move_forward
0	23	(7, 10)	s	moving	move_forward
0	24	(8, 10)	s	moving	move_forward
0	25	(9, 10)	s	moving	move_right
0	26	(9, 9)	w	moving	move_forward
0	27	(9, 8)	w	moving	move_left
0	28	(10, 8)	s	moving	move_forward
0	29	(10, 7)	w	moving	move_forward
0	30	(10, 6)	w	moving	move_forward
0	31	(10, 5)	w	moving	none

Table 2. Path taken by agent 1 in the baseline encoding

agent	timestep	position	direction	status	given_command
1	1	(9, 5)	e	waiting	move_forward
1	2	(9, 6)	e	ready to depart	move_forward
1	3	(9, 7)	e	moving	move_forward
1	4	(9, 8)	e	moving	move_forward
1	5	(9, 9)	e	moving	move_forward
1	6	(9, 10)	e	moving	move_left
1	7	(8, 10)	n	moving	move_forward
1	8	(7, 10)	n	moving	move_forward
1	9	(6, 10)	n	moving	move_forward
1	10	(6, 11)	e	moving	move_forward
1	11	(6, 12)	e	moving	move_forward
1	12	(6, 13)	e	moving	move_forward
1	13	(6, 14)	e	moving	move_forward
1	14	(6, 15)	e	moving	move_forward
1	15	(6, 16)	e	moving	move_right
1	16	(7, 16)	s	moving	move_forward
1	17	(7, 17)	e	moving	move_forward
1	18	(7, 18)	e	moving	move_forward
1	19	(7, 19)	e	moving	none

B Tables for Energy Encoding

Table 3. Path taken by agent 0 in the energy encoding

agent	timestep	position	direction	status	given_command	temp_speed	energy
0	1	(6, 19)	w	waiting	move_forward	4	none
0	5	(6, 18)	w	ready to depart	move_forward	3	12
0	8	(6, 17)	w	moving	move_forward	2	22
0	10	(6, 16)	w	moving	move_forward	1	50
0	11	(6, 15)	w	moving	move_forward	2	200
0	13	(6, 14)	w	moving	move_right	3	50
0	16	(5, 14)	n	moving	move_forward	2	24
0	18	(4, 14)	n	moving	move_forward	3	50
0	21	(4, 13)	w	moving	move_forward	2	24
0	23	(4, 12)	w	moving	move_forward	1	50
0	24	(4, 11)	w	moving	move_forward	1	200
0	25	(4, 10)	w	moving	move_forward	1	200
0	26	(4, 9)	w	moving	move_forward	1	200
0	27	(4, 8)	w	moving	move_forward	1	200
0	28	(4, 7)	w	moving	move_forward	1	200
0	29	(4, 6)	w	moving	move_forward	1	200
0	30	(4, 5)	w	moving	move_forward	1	200
0	31	(4, 4)	w	moving	move_forward	1	200
0	32	(4, 3)	w	moving	move_forward	1	200
0	33	(4, 2)	w	moving	move_forward	1	200
0	34	(4, 1)	w	moving	move_forward	2	200
0	36	(4, 0)	w	moving	move_forward	3	50
0	39	(5, 0)	s	moving	move_forward	2	24
0	41	(6, 0)	s	moving	move_forward	1	50
0	42	(7, 0)	s	moving	move_forward	1	200
0	43	(8, 0)	s	moving	move_forward	2	200
0	45	(9, 0)	s	moving	move_forward	3	50
0	48	(9, 1)	e	moving	move_forward	2	24
0	50	(9, 2)	e	moving	move_right	3	50
0	53	(10, 2)	s	moving	move_forward	3	24
0	56	(10, 3)	e	moving	move_forward	3	24
0	59	(10, 4)	e	moving	move_forward	4	22
0	63	(10, 5)	e	moving	none	4	12

Table 4. Path taken by agent 1 in the energy encoding

agent	timestep	position	direction	status	given_command	temp_speed	energy
1	1	(9, 5)	e	waiting	wait	4	none
1	2	(9, 5)	e	ready to depart	move_forward	4	-7
1	6	(9, 6)	e	moving	move_forward	3	18
1	9	(9, 7)	e	moving	move_forward	4	33
1	13	(9, 8)	e	moving	move_forward	3	18
1	16	(9, 9)	e	moving	wait	4	33
1	17	(9, 9)	e	stopped	wait	4	-7
1	18	(9, 9)	e	stopped	move_forward	4	none
1	22	(9, 10)	e	moving	move_left	4	18
1	26	(8, 10)	n	moving	wait	4	19
1	27	(8, 10)	n	stopped	move_forward	4	-7
1	31	(7, 10)	n	moving	move_forward	3	18
1	34	(6, 10)	n	moving	move_forward	4	33
1	38	(6, 11)	e	moving	move_forward	3	19
1	41	(6, 12)	e	moving	move_forward	2	33
1	43	(6, 13)	e	moving	move_forward	2	75
1	45	(6, 14)	e	moving	move_forward	2	75
1	47	(6, 15)	e	moving	move_forward	3	75
1	50	(6, 16)	e	moving	move_right	3	33
1	53	(7, 16)	s	moving	move_forward	3	36
1	56	(7, 17)	e	moving	move_forward	3	36
1	59	(7, 18)	e	moving	move_forward	4	33
1	63	(7, 19)	e	moving	none	4	18